FIAP - Faculdade de Informática e Administração Paulista

Pós-Graduação em Arquitetura e Desenvolvimento Java

Jefferson Ricardo dos Santos

RM371825

SISTEMA DE GERENCIAMENTO DE USUÁRIOS PARA RESTAURANTES: DESENVOLVIMENTO DE API RESTFUL COM SPRING BOOT

Tech Challenge — Fase 1

São Paulo

2026

Jefferson Ricardo dos Santos

SISTEMA DE GERENCIAMENTO DE USUÁRIOS PARA RESTAURANTES: DESENVOLVIMENTO DE API RESTFUL COM SPRING BOOT

Trabalho apresentado à FIAP - Faculdade de Informática e Administração Paulista, como parte dos requisitos para aprovação na disciplina Tech Challenge — Fase 1 do curso de Pós-Graduação em Arquitetura e Desenvolvimento Java.

São Paulo

2026

RESUMO

Este trabalho apresenta o desenvolvimento de uma API RESTful para gerenciamento de usuários, construída como parte do Tech Challenge — Fase 1 do curso de Pós-Graduação em Arquitetura e Desenvolvimento Java da FIAP. O sistema foi projetado para atender a um grupo de restaurantes que necessitam de uma plataforma compartilhada de gestão, permitindo o cadastro, atualização, exclusão e autenticação de dois tipos de usuários: donos de restaurante e clientes. A aplicação foi desenvolvida utilizando Java 17 e Spring Boot 3.2.3, seguindo uma arquitetura em camadas com padrões de projeto como DTO Pattern, Repository Pattern e Service Layer. O tratamento de erros segue o padrão ProblemDetail (RFC 7807), e a segurança das senhas é garantida pelo algoritmo BCrypt. A persistência de dados utiliza PostgreSQL como banco relacional, e toda a infraestrutura é orquestrada via Docker Compose. O projeto conta com mais de 100 testes automatizados (JUnit 5 + Mockito), documentação interativa via Swagger/OpenAPI e coleção Postman com 50 cenários de teste. Como funcionalidades extras, além do escopo obrigatório, foram implementados: testes unitários e de integração automatizados, deploy em produção no Render.com e CI/CD com GitHub Actions.

Palavras-chave: API RESTful. Spring Boot. Java. Gerenciamento de Usuários. Docker. PostgreSQL.

SUMÁRIO

1. Introdução
2. Objetivos
3. Fundamentação Teórica
4. Metodologia
5. Desenvolvimento
6. Resultados e Discussão
7. Conclusão
8. Referências
9. Apêndices

---

1. INTRODUÇÃO

Na região de São Paulo, um grupo de restaurantes identificou a necessidade de um sistema de gestão compartilhado para seus estabelecimentos. O alto custo de sistemas individuais motivou a união desses restaurantes para o desenvolvimento de uma plataforma única, que permita aos clientes escolherem restaurantes com base na comida oferecida, e não na qualidade do sistema de gestão.

O presente trabalho corresponde à primeira fase desse sistema, focada no módulo de gerenciamento de usuários. Esta fase estabelece a base para as funcionalidades futuras, como cadastro de restaurantes, cardápios, avaliações e pedidos online.

A entrega do sistema foi planejada em fases, garantindo uma implementação gradual e controlada, com ajustes e melhorias contínuas conforme o sistema for sendo utilizado e avaliado.

1.1 Justificativa

O gerenciamento de usuários é o alicerce de qualquer sistema multiusuário. Sem um módulo robusto de cadastro, autenticação e controle de perfis, as funcionalidades subsequentes (restaurantes, pedidos, avaliações) não podem ser implementadas de forma segura. A escolha por iniciar o desenvolvimento por este módulo garante que a base do sistema esteja sólida antes de avançar para funcionalidades mais complexas.

1.2 Escopo

Este trabalho abrange exclusivamente o módulo de gerenciamento de usuários, incluindo:

- Cadastro, atualização e exclusão de usuários (CRUD completo)
- Dois tipos de usuário: Dono de Restaurante e Cliente
- Validação de login por meio de serviço dedicado
- Troca de senha em endpoint separado
- Busca de usuários por nome
- Garantia de unicidade do e-mail
- Registro automático da data da última alteração
- Dockerização completa com Docker Compose
- Documentação com Swagger/OpenAPI
- Coleção Postman para testes

---

2. OBJETIVOS

2.1 Objetivo Geral

Desenvolver um backend completo e robusto utilizando Spring Boot e os princípios aprendidos na Fase 1 do curso, implementando uma API RESTful para gerenciamento de usuários de um sistema compartilhado de restaurantes.

2.2 Objetivos Específicos

- Implementar CRUD completo de usuários com validação de dados
- Criar dois tipos de usuário obrigatórios: Dono de Restaurante e Cliente
- Desenvolver serviço de validação de login (autenticação por login e senha)
- Implementar endpoint separado para troca de senha
- Implementar endpoint distinto para atualização de dados do usuário
- Garantir unicidade do e-mail no cadastro
- Implementar busca de usuários por nome (parcial, case-insensitive)
- Registrar automaticamente a data da última alteração
- Implementar versionamento de API
- Adotar o padrão ProblemDetail (RFC 7807) para respostas de erro
- Dockerizar a aplicação com Docker Compose e banco relacional PostgreSQL

- Documentar a API com Swagger/OpenAPI
- Criar coleção Postman cobrindo cenários de sucesso e erro
- Implementar testes automatizados com JUnit 5 e Mockito (desafio extra)

---

3. FUNDAMENTAÇÃO TEÓRICA

3.1 Arquitetura REST

REST (Representational State Transfer) é um estilo arquitetural para sistemas distribuídos, proposto por Roy Fielding em sua tese de doutorado (2000). APIs RESTful utilizam os métodos HTTP (GET, POST, PUT, PATCH, DELETE) para operações sobre recursos identificados por URIs, seguindo princípios como statelessness, interface uniforme e representação de recursos.

3.2 Spring Boot

O Spring Boot é um framework Java que simplifica a criação de aplicações Spring, oferecendo configuração automática, servidor embutido e um ecossistema completo de módulos (Web, Data JPA, Security, Actuator). A versão 3.2.3, utilizada neste projeto, é baseada no Spring Framework 6 e requer Java 17 como versão mínima.

3.3 Padrão ProblemDetail (RFC 7807)

A RFC 7807 define um formato padronizado para representar erros em APIs HTTP, utilizando os campos `type`, `title`, `status`, `detail` e `instance`. O Spring Framework 6 oferece suporte nativo a este padrão através da classe `ProblemDetail`, adotada neste projeto para todas as respostas de erro.

3.4 BCrypt

BCrypt é um algoritmo de hash adaptativo baseado no cifrador Blowfish, projetado especificamente para armazenamento seguro de senhas. Seu fator de custo ajustável permite aumentar a complexidade computacional conforme o hardware evolui, tornando ataques de força bruta progressivamente mais custosos.

3.5 Docker e Docker Compose

Docker é uma plataforma de containerização que empacota aplicações e suas dependências em containers isolados e portáveis. Docker Compose é

uma ferramenta para definir e executar aplicações multi-container, permitindo orquestrar serviços (aplicação + banco de dados) com um único arquivo de configuração.

3.6 Padrões de Projeto Utilizados

- **DTO Pattern (Data Transfer Object):** separação entre a camada de apresentação e o domínio, utilizando Java Records imutáveis para transferência de dados.
- **Repository Pattern:** abstração do acesso a dados, implementada via Spring Data JPA com queries derivadas do nome do método.
- **Service Layer:** centralização da lógica de negócio em uma camada dedicada, desacoplada do controller e do repository.
- **Global Exception Handler:** tratamento centralizado de exceções com `@RestControllerAdvice`, garantindo respostas de erro consistentes em toda a API.

---

4. METODOLOGIA

4.1 Processo de Desenvolvimento

O projeto foi desenvolvido seguindo uma abordagem iterativa, organizada em sprints com tasks definidas em um backlog. O gerenciamento foi feito via GitHub Projects (Kanban), com issues rastreando cada funcionalidade.

4.2 Fluxo de Trabalho Git

Foi adotado o modelo de branching com três branches principais:

- **main:** código em produção, recebe merges apenas da develop
- **develop:** branch de integração, recebe merges das feature branches
- **feature/*:** branches individuais para cada task

Cada funcionalidade foi desenvolvida em uma feature branch, revisada via Pull Request e integrada à develop antes de ser promovida à main.

4.3 Ambientes

Ambiente	Banco de Dados	Profile Spring	Execução
Desenvolvimento	PostgreSQL 15 (Docker)	`dev`	`docker-compose up`

Testes	H2 em memória	`test`	`mvn test`
Produção	PostgreSQL (Render.com)	`prod`	Deploy automático

4.4 Ferramentas

Ferramenta	Finalidade
IntelliJ IDEA	IDE de desenvolvimento
Maven	Build e gerenciamento de dependências
Docker / Docker Compose	Containerização e orquestração
GitHub Actions	CI/CD (build, testes, deploy)
SonarCloud	Análise estática de código
Postman / Bruno	Testes manuais de API
Swagger UI	Documentação interativa
Render.com	Hospedagem em produção

4.5 Desafios Encontrados

- **JaCoCo e Java 25:** como a máquina local roda Java 25 e o JaCoCo 0.8.11 só suporta até Java 21, os testes geravam warnings. Resolvi com a flag `net.bytebuddy.experimental=true` no Surefire.
- **Conflito de formatação na TASK-037:** houve conflitos de branches durante o desenvolvimento em equipe — um commit veio com o auto-format da IDE ligado, o que bagunçou o diff e removeu documentação Swagger de outros endpoints. Foi necessário restaurar manualmente. Além disso, durante a resolução de conflitos entre branches, houve perda de conteúdo que só foi percebida após o merge, exigindo revisão cuidadosa dos arquivos afetados.
- **SonarCloud:** a configuração dos tokens, quality gates e integração com GitHub Actions levou mais tempo do que o esperado.
- **Git:** em um momento fiz `git pull` na branch errada e tive que abortar o merge. Serviu de lição para sempre checar a branch antes.
- **Deploy no Render:** configurar o build automático no Render deu trabalho. As variáveis de ambiente do banco de dados precisavam bater com o profile `prod`, o Dockerfile teve ajustes para funcionar no ambiente do Render, e o primeiro deploy falhava por timeout até acertar a porta via variável `$PORT`. Além disso, a instância gratuita

entra em sleep após inatividade, o que confundia na hora de testar se o deploy tinha funcionado.

---

5. DESENVOLVIMENTO

5.1 Arquitetura da Aplicação

A aplicação segue uma arquitetura em camadas (Layered Architecture), onde cada camada possui responsabilidades bem definidas:

```
┌──────────────────────────────────────────────────────────┐
│                    CLIENT (HTTP)                         │
└──────────────────────────────────────────────────────────┘
                          │
                          ▼
┌──────────────────────────────────────────────────────────┐
│              Controller (REST API)                       │
│    Recebe requisições HTTP, valida entrada,              │
│    delega para o Service e retorna ResponseEntity        │
└──────────────────────────────────────────────────────────┘
                          │
                          ▼
┌──────────────────────────────────────────────────────────┐
│                Service (Negócio)                         │
│    Regras de negócio, validações de unicidade,           │
│    hash de senha com BCrypt, orquestração                │
└──────────────────────────────────────────────────────────┘
                          │
                          ▼
┌──────────────────────────────────────────────────────────┐
│              Repository (Dados)                          │
│    Acesso ao banco via Spring Data JPA                   │
│    Queries derivadas do nome do método                   │
└──────────────────────────────────────────────────────────┘
                          │
                          ▼
┌──────────────────────────────────────────────────────────┐
│                        PostgreSQL  (Banco  Relacional)
│
└──────────────────────────────────────────────────────────┘
```

5.2 Tecnologias Utilizadas

Tecnologia	Versão	Finalidade
Java	17 LTS	Linguagem principal
Spring Boot	3.2.3	Framework web
Spring Data JPA	3.2.3	Persistência de dados

Spring Security	3.2.3	BCrypt para hash de senhas
PostgreSQL	15	Banco de dados relacional
H2	Runtime	Banco em memória para testes
Springdoc OpenAPI	2.3.0	Documentação Swagger
Docker / Docker Compose	3.9	Containerização e orquestração
JUnit 5 + Mockito	-	Testes automatizados
JaCoCo	0.8.11	Cobertura de testes
Maven	3.9+	Build e dependências

5.3 Estrutura do Projeto

```
src/
├── main/java/com/fiap/fase1/
│   ├── Fase1SpringExplorerApplication.java
│   ├── config/
│   │   ├── OpenApiConfig.java
│   │   └── SecurityConfig.java
│   ├── controller/
│   │   └── UserController.java
│   ├── service/
│   │   └── UserService.java
│   ├── repository/
│   │   └── UserRepository.java
│   ├── model/
│   │   ├── User.java
│   │   └── UserType.java
│   ├── dto/
│   │   ├── UserRequestDTO.java
│   │   ├── UserUpdateDTO.java
│   │   ├── UserResponseDTO.java
│   │   ├── ChangePasswordDTO.java
│   │   ├── LoginRequestDTO.java
│   │   └── LoginResponseDTO.java
│   ├── exception/
│   │   ├── GlobalExceptionHandler.java
│   │   ├── UserNotFoundException.java
│   │   ├── EmailAlreadyExistsException.java
│   │   ├── LoginAlreadyExistsException.java
│   │   └── InvalidCredentialsException.java
│   └── validation/
│       ├── ValidPassword.java
│       ├── PasswordValidator.java
│       ├── SafeInput.java
│       └── SafeInputValidator.java
```

```
├── main/resources/
│   ├── application.properties
│   ├── application-dev.properties
│   └── application-prod.properties
└── test/
    ├── java/com/fiap/fase1/
    │   ├── controller/UserControllerTest.java
    │   ├── service/UserServiceTest.java
    │   ├── repository/UserRepositoryTest.java
    │   ├── model/UserModelTest.java
    │   ├── integration/UserIntegrationTest.java
    │   └── validation/
    │       ├── PasswordValidatorTest.java
    │       └── SafeInputValidatorTest.java
    └── resources/
        └── application-test.properties
```

5.4 Modelagem de Dados

5.4.1 Entidade User

A aplicação possui uma única entidade `User`, mapeada para a tabela `users` no banco de dados:

```
@Entity
@Table(name = "users")
public class User {
                @Id    @GeneratedValue(strategy    =
GenerationType.IDENTITY)
    private Long id;

    @NotBlank @Size(min = 2, max = 100)
    @Column(nullable = false)
    private String name;

    @Email @NotBlank
    @Column(nullable = false, unique = true)
    private String email;

    @NotBlank @Size(min = 3, max = 50)
    @Column(nullable = false, unique = true)
    private String login;

    @NotBlank @Size(min = 8)
    @Column(nullable = false)
    private String password;

    @NotBlank @Size(max = 255)
    @Column(nullable = false)
```

```
    private String address;

    @NotNull @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private UserType type;

    @NotNull @Column(nullable = false)
    private LocalDateTime lastModifiedDate;
}
```

5.4.2 Tipos de Usuário

O sistema implementa obrigatoriamente dois tipos de usuário, representados pelo enum `UserType`:

- `RESTAURANT_OWNER` — Dono de restaurante
- `CUSTOMER` — Cliente

5.4.3 Diagrama Entidade-Relacionamento

```
┌──────────────────────────────────────┐
│              users                   │
├──────────────────────────────────────┤
│ id                  BIGSERIAL (PK)   │
│ name                VARCHAR(100) NN  │
│ email               VARCHAR(255) NN UQ│
│ login               VARCHAR(50)  NN UQ│
│ password            VARCHAR(255) NN  │
│ address             VARCHAR(255) NN  │
│ type                VARCHAR(20)  NN  │
│ last_modified_date  TIMESTAMP    NN  │
└──────────────────────────────────────┘
```

5.4.4 DDL

```
CREATE TABLE users (
    id BIGSERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(255) NOT NULL UNIQUE,
    login VARCHAR(50) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    address VARCHAR(255) NOT NULL,
    type VARCHAR(20) NOT NULL,
    last_modified_date TIMESTAMP NOT NULL
);
```

5.4.5 Data Transfer Objects (DTOs)

DTO	Finalidade	Campos
`UserRequestDTO`	Criação de usuário	name, email, login, password, address, type
`UserUpdateDTO`	Atualização (sem senha)	name, email, login, address, type
`UserResponseDTO`	Retorno ao cliente	id, name, email, login, address, type, lastModifiedDate
`ChangePasswordDTO`	Troca de senha	currentPassword, newPassword
`LoginRequestDTO`	Requisição de login	login, password
`LoginResponseDTO`	Resposta de login	message, userId, login, email, lastModifiedDate

5.5 Endpoints da API

5.5.1 Versionamento

Todos os endpoints utilizam versionamento por URL: `/api/v1/usuarios`.

5.5.2 Tabela de Endpoints

Método	Endpoint	Descrição	Sucesso	Erro
POST	`/api/v1/usuarios`	Criar usuário	201	400, 409
GET	`/api/v1/usuarios`	Listar todos	200	-
GET	`/api/v1/usuarios?name={nome}`	Buscar por nome	200	-
GET	`/api/v1/usuarios/{id}`	Buscar por ID	200	404
PUT	`/api/v1/usuarios/{id}`	Atualizar dados	200	400, 404, 409

DELETE	`/api/v1/usuarios/{id}`	Deletar usuário	204	404
POST	`/api/v1/usuarios/login`	Validar login	200	400, 401
PATCH	`/api/v1/usuarios/{id}/password`	Trocar senha	200	400, 404

5.5.3 Exemplos de Requisições e Respostas

Criar Usuário (CUSTOMER) — 201 Created:

```
POST /api/v1/usuarios
Content-Type: application/json

{
  "name": "João Silva",
  "email": "joao.silva@email.com",
  "login": "joaosilva",
  "password": "Senha123",
  "address": "Rua das Flores, 123 - São Paulo/SP",
  "type": "CUSTOMER"
}
```

```
{
  "id": 1,
  "name": "João Silva",
  "email": "joao.silva@email.com",
  "login": "joaosilva",
  "address": "Rua das Flores, 123 - São Paulo/SP",
  "type": "CUSTOMER",
  "lastModifiedDate": "2026-04-23T10:30:00"
}
```

Criar Usuário (RESTAURANT_OWNER) — 201 Created:

```
POST /api/v1/usuarios
Content-Type: application/json

{
  "name": "Maria Oliveira",
  "email": "maria.oliveira@restaurante.com",
  "login": "mariaoliveira",
  "password": "Restaurante1",
```

```
  "address": "Av. Paulista, 1000 - São Paulo/SP",
  "type": "RESTAURANT_OWNER"
}
```

Email Duplicado — 409 Conflict:

```
{
  "type": "https://api.fiap.com/errors/conflict",
  "title": "Email já cadastrado",
  "status": 409,
          "detail":     "Email     já     cadastrado:
joao.silva@email.com",
  "instance": "/api/v1/usuarios"
}
```

Dados Inválidos — 400 Bad Request:

```
{
  "type": "https://api.fiap.com/errors/validation",
  "title": "Dados inválidos",
  "status": 400,
  "detail": "Erro de validação",
  "instance": "/api/v1/usuarios",
  "campos": {
    "name": "O nome é obrigatório",
    "email": "O email é obrigatório",
    "password": "A senha é obrigatória"
  }
}
```

Buscar por Nome — 200 OK:

```
GET /api/v1/usuarios?name=João
```

A busca é parcial e case-insensitive. Retorna lista vazia (200) se nenhum resultado for encontrado.

Login — 200 OK:

```
POST /api/v1/usuarios/login
Content-Type: application/json
```

```
{ "login": "joaosilva", "password": "Senha123" }
```

```
{
  "message": "Login realizado com sucesso",
  "userId": 1,
  "login": "joaosilva",
  "email": "joao.silva@email.com",
  "lastModifiedDate": "2026-04-23T10:30:00"
}
```

Login Inválido — 401 Unauthorized:

```
{
  "type": "https://api.fiap.com/errors/unauthorized",
  "title": "Credenciais inválidas",
  "status": 401,
  "detail": "Login ou senha inválidos",
  "instance": "/api/v1/usuarios/login"
}
```

Trocar Senha — 200 OK:

```
PATCH /api/v1/usuarios/1/password
Content-Type: application/json

{   "currentPassword":   "Senha123",   "newPassword":
"NovaSenha456" }
```

```
{ "mensagem": "Senha alterada com sucesso" }
```

5.5.4 Tratamento de Erros (ProblemDetail — RFC 7807)

Todas as respostas de erro seguem o padrão ProblemDetail:

Campo	Descrição
`type`	URI que identifica o tipo de erro
`title`	Título legível do erro

`status`	Código HTTP
`detail`	Descrição detalhada
`instance`	URI do recurso que gerou o erro
`campos`	Mapa campo → mensagem (apenas em erros de validação)

5.5.5 Regras de Validação

Campo	Regras
`name`	Obrigatório, 2-100 caracteres, sem caracteres perigosos
`email`	Obrigatório, formato email válido, único no sistema
`login`	Obrigatório, 3-50 caracteres, único no sistema
`password`	Mínimo 8 caracteres, 1 maiúscula, 1 minúscula, 1 número
`address`	Obrigatório, máximo 255 caracteres
`type`	Obrigatório: `CUSTOMER` ou `RESTAURANT_OWNER`

5.6 Documentação Swagger

A documentação interativa da API é gerada automaticamente pelo Springdoc OpenAPI 2.3.0 e está disponível em:

- **Local:** http://localhost:8080/swagger-ui.html
- **Produção:** https://one2adjt-fase-1.onrender.com/swagger-ui.html

Todos os endpoints possuem anotações `@Operation`, `@ApiResponses`, `@Parameter` e `@Schema` com exemplos de requisição e resposta. Os endpoints são organizados em dois grupos: **Usuários** (CRUD) e **Autenticação** (login e troca de senha).

5.7 Coleção Postman

O arquivo da coleção está em `docs/api-collection/fiap-fase1-usuarios.json` e contém 50 requests organizados em duas pastas (`local` e `prod`), cada uma com subpastas Usuários, Autenticação e Health Check.

Cenários cobertos:

Cenário	Requests

Cadastro de usuário válido (CUSTOMER e RESTAURANT_OWNER)	2
Cadastro inválido (campos vazios, email inválido, senha fraca, duplicados)	6
Listagem e busca por nome (parcial, case-insensitive, inexistente)	4
Busca por ID (sucesso e inexistente)	2
Atualização (sucesso, ID inexistente, dados inválidos)	3
Deleção (sucesso e ID inexistente)	2
Login (sucesso, inexistente, senha incorreta, campos vazios)	4
Troca de senha (sucesso, senha incorreta, senha fraca, ID inexistente)	4
Health Check	1

Cada request possui scripts de teste Postman que validam status code, estrutura do response e valores dos campos.

5.8 Infraestrutura Docker

5.8.1 Dockerfile (Multi-stage Build)

```
# Build Stage
FROM maven:3.9.6-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline
COPY src ./src
RUN mvn clean package -DskipTests

# Runtime Stage
FROM eclipse-temurin:17-jre-jammy
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
ENV PORT=8080
EXPOSE 8080
ENTRYPOINT ["sh", "-c", "java -jar app.jar --
server.port=$PORT"]
```

5.8.2 Docker Compose

```yaml
version: "3.9"
services:
  postgres:
    image: postgres:15
    environment:
      POSTGRES_DB: fiap_fase1
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
    ports:
      - "5433:5432"
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 10s
      timeout: 5s
      retries: 5

  app:
    build: .
    environment:
      SPRING_PROFILES_ACTIVE: dev
      SERVER_PORT: 8080
    ports:
      - "8080:8080"
    depends_on:
      postgres:
        condition: service_healthy
```

5.8.3 Variáveis de Ambiente

Variável	Valor	Descrição
`POSTGRES_DB`	`fiap_fase1`	Nome do banco de dados
`POSTGRES_USER`	`postgres`	Usuário do banco
`POSTGRES_PASSWORD`	`postgres`	Senha do banco
`SPRING_PROFILES_ACTIVE`	`dev`	Profile Spring Boot
`SERVER_PORT`	`8080`	Porta da aplicação

5.8.4 Passo a Passo para Execução

1. Clonar o repositório:

```
git clone https://github.com/jeffesa/12ADJT-fase-1.git
cd 12ADJT-fase-1
```

2. Subir a aplicação completa:

```
docker-compose up
```

3. Verificar se está rodando:

```
curl http://localhost:8080/actuator/health
```

4. Acessar:

Recurso	URL
API	http://localhost:8080/api/v1/usuarios
Swagger UI	http://localhost:8080/swagger-ui.html
Health Check	http://localhost:8080/actuator/health

5. Parar:

```
docker-compose down
```

---

6. RESULTADOS E DISCUSSÃO

6.1 Funcionalidades Implementadas

Todos os requisitos especificados no enunciado do Tech Challenge foram atendidos:

Requisito	Status	Implementação
Cadastro, atualização e exclusão de usuários	✅	POST, PUT, DELETE `/api/v1/usuarios`

Dois tipos de usuário (dono de restaurante e cliente)	✅	Enum `UserType`
Troca de senha em endpoint separado	✅	PATCH `/api/v1/usuarios/{id}/password`
Atualização de dados em endpoint distinto do de senha	✅	PUT `/api/v1/usuarios/{id}`
Registro da data da última alteração	✅	`@PrePersist` / `@PreUpdate`
Busca de usuários pelo nome	✅	GET `/api/v1/usuarios?name=`
Garantia de unicidade do e-mail	✅	`@Column(unique=true)` + validação no service
Validação de login obrigatória	✅	POST `/api/v1/usuarios/login`
Versionamento de API	✅	`/api/v1/`
ProblemDetail (RFC 7807)	✅	`GlobalExceptionHandler`
Dockerização com Docker Compose	✅	`docker-compose.yml` + `Dockerfile`
Banco relacional (PostgreSQL)	✅	PostgreSQL 15
Documentação Swagger/OpenAPI	✅	Springdoc OpenAPI 2.3.0
Coleção Postman	✅	50 requests com testes

6.2 Testes Automatizados (Desafio Extra)

O projeto conta com **104+ testes automatizados**, cobrindo todas as camadas:

Classe de Teste	Tipo	Quantidade
`UserControllerTest`	Unitário (MockMvc)	24

`UserServiceTest`	Unitário (Mockito)	20
`UserRepositoryTest`	Integração (DataJpaTest)	10
`UserModelTest`	Unitário	8
`UserIntegrationTest`	Integração (SpringBootTest)	25
`PasswordValidatorTest`	Unitário	8
`SafeInputValidatorTest`	Unitário	9
Total		**104+**

6.3 Deploy em Produção (Funcionalidade Extra)

Como funcionalidade extra, não exigida no enunciado do Tech Challenge, a aplicação foi deployada no Render.com, permitindo acesso público para demonstração e testes:

Recurso	URL
API Base	https://one2adjt-fase-1.onrender.com
Swagger UI	https://one2adjt-fase-1.onrender.com/swagger-ui.html
Health Check	https://one2adjt-fase-1.onrender.com/actuator/health

6.4 Repositório

O código-fonte está disponível em repositório público no GitHub:

https://github.com/jeffesa/12ADJT-fase-1

O repositório contém: código-fonte, README, documentação Swagger, coleção Postman e este relatório técnico.

---

7. CONCLUSÃO

O presente trabalho atingiu todos os objetivos propostos, entregando uma API RESTful completa e funcional para gerenciamento de usuários. A aplicação implementa todas as funcionalidades exigidas pelo Tech Challenge — Fase 1, incluindo CRUD de usuários, autenticação, troca de senha em endpoint separado, busca por nome, unicidade de e-mail e dockerização completa.

Além dos requisitos obrigatórios, foram implementadas as seguintes funcionalidades extras, não exigidas no enunciado: testes automatizados com JUnit 5 e Mockito (104+ testes), deploy em produção no Render.com, CI/CD com GitHub Actions e análise de código com SonarCloud.

A arquitetura em camadas, combinada com padrões de projeto como DTO Pattern, Repository Pattern e Global Exception Handler, resultou em um código organizado, testável e de fácil manutenção. O uso do padrão ProblemDetail (RFC 7807) garante respostas de erro consistentes e padronizadas.

O sistema está preparado para receber as funcionalidades das próximas fases (cadastro de restaurantes, cardápios, avaliações e pedidos), uma vez que a base de gerenciamento de usuários está sólida e bem testada.

---

8. REFERÊNCIAS

FIELDING, Roy Thomas. **Architectural Styles and the Design of Network-based Software Architectures.** 2000. Tese (Doutorado) — University of California, Irvine, 2000.

SPRING. **Spring Boot Reference Documentation.** Disponível em: https://docs.spring.io/spring-boot/docs/3.2.3/reference/html/. Acesso em: abr. 2026.

SPRING. **Spring Data JPA Reference Documentation.** Disponível em: https://docs.spring.io/spring-data/jpa/docs/current/reference/html/. Acesso em: abr. 2026.

NOTTINGHAM, M.; WILDE, E. **RFC 7807 — Problem Details for HTTP APIs.** Internet Engineering Task Force (IETF), 2016. Disponível em: https://tools.ietf.org/html/rfc7807. Acesso em: abr. 2026.

DOCKER. **Docker Documentation.** Disponível em: https://docs.docker.com/. Acesso em: abr. 2026.

POSTGRESQL. **PostgreSQL Documentation.** Disponível em: https://www.postgresql.org/docs/15/. Acesso em: abr. 2026.

OPENAPI INITIATIVE. **OpenAPI Specification.** Disponível em: https://spec.openapis.org/oas/v3.0.3. Acesso em: abr. 2026.

---

9. APÊNDICES

Apêndice A — Links do Projeto

Recurso	URL
Repositório GitHub	https://github.com/jeffesa/12ADJT-fase-1
Aplicação em Produção	https://one2adjt-fase-1.onrender.com
Swagger UI (Produção)	https://one2adjt-fase-1.onrender.com/swagger-ui.html
Coleção Postman	`docs/api-collection/fiap-fase1-usuarios.json`

Apêndice B — Como Importar a Coleção Postman

1. Abra o Postman
2. Clique em **Import** > **Upload Files**
3. Selecione `docs/api-collection/fiap-fase1-usuarios.json`
4. A coleção será importada com variáveis `localUrl` e `prodUrl` pré-configuradas

---

Trabalho apresentado em maio de 2026.